Statement of Work
Progetto
TasKCal

Representative	
Version	1.0
Data	10/10/2026
Addressed to	Professor of Software Engineering 2026/27
Presented by	Fabiana GODONO, Amal BENNINI, Francesco BOLOGNA, Muhammad Shafin AHAMMED
Approved by	

Revision History

Date	Version	Description	Authors

Statement of Work (SOW) for project TasKCal

1. System Scope

1.1 Problem Statement

Students and busy individuals often struggle with fragmented schedules and distraction-heavy productivity apps. Most keep track of their lectures, exams, deadlines, group meetings and personal commitments on a mix of their apps, calendar notes and their own memory. Many of the existing apps are either too basic (no task tracker) or too heavy (mandatory accounts, cloud sync, ads, subscription and constant internet use). The result is not using the apps from being too overwhelmed, leading to missed deadlines and double-booked days.

1.2 Goal

TasKCal is a simple local Android app which keeps events and tasks in one place, on the users' phone. No ads, no subscription, no external database to keep syncing their app to. It is aimed at students and other people who want to stay organised without a complicated tool. This first version (1.0) covers only the core features required for calendar and task management; everything else will be added later, once the core work.

1.3 Scopes

What the System **must** do (in scope):

- **Event management:** users can *create*, *edit*, *delete* and *view* events. Each event can include a title, a date, a start and an end time, an all day option, a description and a colour
- **Calendar views:** the application provides a daily view, displaying the events scheduled for the selected day, and a monthly view. Users can return to the current date using a "Today" action.
- **Reminder:** users can set local reminders for events and tasks. The reminder time can be configured through the user interface.
- **Task tab:** the application provides a separate screen containing a to-do list. Each task has a title, an optional due date, a priority level (Urgent, Medium, or Low), and a completion status (completed or not completed). Tasks with a due date will also appear in the calendar view.

- **Local Storage:** all application data will be stored in a local database on the device and will remain available after the application is closed or the device is restarted.

Not in this version (scopes that could be added later):

- **Appearance:** user are able to choose from predefined theme colours.
- **Recurring Events:** Daily, Monthly, Weekly or Yearly repetition with either an end date or a number of occurrences.
- **Export and Import of data:** If time allows after the core is done.
- **Grouping multiple activities together.**

2. Project Timeline & Milestones

Project Start date: 12 October 2026

Project End date: January 2027 (final delivery on 20/01/2027, to be confirmed with the instructor)

3. Deliverables

- Project plan, Requirement Analysis Document (RAD), System design Document (SDD), Object Design Document (ODD) summary, Traceability matrix, Test documents, Source code, Installable app, User guide.

4. Constraints

4.1 Collaboration and communication constraints.

- Respect all deadlines that are **defined in** the **statement of work**
- All source codes and documentations live on **GitHub** repository, and every member contributes through their own commits
- Work is divided into tasks on shared board (Trello), every task has an owner and a status.

- Using traceable communication tool (Discord) for team communications. Decisions are written down there (**#Decisions**).
- Work is merged through **pull-request**, never pushed directly to the *main* branch.

4.2 Technical Constraints

Platform and Technology

- Native Android app written in Java, minimum Android 8.0 (API 26)
- Developed in **Android Studio.**
- User interface built with **XML layouts**.
- Local storage with **Room** (SQLite). No backend server, no network permission needed for core features.
- Reminders implemented with the Android notification and alarm APIs.

Analisi e specifica dei requisiti

- **Functional** and **non-functional** requirements are both specified.
- Each team member writes **minimum 2** and **maximum 4** scenarios;
- Each team member must write **minimum 2** and **maximum 4** functional and non-functional requirements;
- **Exactly** One use-case for each team member – the added use cases will **not** be evaluated;
- **Exactly** One sequence diagram per 2 team members – additional sequence diagrams will **not** be evaluated;
- **Exactly** One statechart per 2 team members – additional diagrams will **not** be evaluated;
- **Exactly** One activity diagram for each team member – additional diagrams will **not** be evaluated;
- Specify **One** class diagram per team - additional object diagrams will **not** be evaluated.

System Design

- Each team member specifies minimum 2 and maximum 4 design goals (for example, offline availability, responsiveness, ease of use etc). The team analyses the trade-off between **at least two pairs** of design goals.
- Definition of a **subsystem decomposition diagram** by team, including a description and rationale for its use.
- Definition of a **Deployment diagram** by team, including a description and rationale for its use.

Object Design

- **Two** design patterns used (or at least designed), chosen from the ones taught in the course;

Notation

- All diagrams use UML;

Testing

- Each student/member performs unit testing of **exactly One** method of a class they developed using category partition.
- Each student/member performs system testing of **exactly One** feature, using category partition.

5. Acceptance Criteria

- Proper use of GitHub, including **adherence to the guidelines established for laboratory activities**.
- Proper use of pull-based development, including **adherence to the guidelines established for laboratory activities**.
- Proper use of Slack/Discord, including **adherence to the guidelines established for laboratory activities**.
- Proper use of Trello, including **adherence to the guidelines established for laboratory activities**.
- Proper Documentation. **Plagiarism detection** tools will be used to identify instances where students have copied from previous years' projects and/or other sources.

- Appropriate unit testing of a developed method, ensuring **compliance with constraints**.
- Appropriate system testing of a feature of the development system, ensuring **compliance with constraints**.

6. Bonus Criteria

- Proper use of **build system** (Gradle);
- Proper use of **continuous integration** (Travis or GitHub Actions)
- Systematic code review: every pull request is reviewed by another member before merging.
- Using advanced testing tools (e.g., **Mockito, Cobertura**, etc.).